Payment Processing for a Shopping System

Week 8 Practice - Question 5

Design: Customer has Orders; Order contains one or more OrderItems referring to Product.
PaymentMethod is an interface;
CreditCardPayment, PayPalPayment, and BankTransferPayment implement it. Failed payment leaves Order Pending.
Java solution:

```
import java.util.*;
class Product {final String name;final double price;Product(String n,double
p){name=n;price=p;}}
class OrderItem {final Product product;final int qty;OrderItem(Product p,int q){if(q<=0)throw
new IllegalArgumentException("Quantity must be positive");product=p;qty=q;}double
total(){return product.price*qty;}}
class Customer {final String name;Customer(String n){name=n;}}
interface PaymentMethod {String name();boolean processPayment(double amount);}
class CreditCardPayment implements PaymentMethod {public String name(){return "Credit
Card";}public boolean processPayment(double a){return true;}}
class PayPalPayment implements PaymentMethod {public String name(){return "PayPal";}public
boolean processPayment(double a){return false;}}
class BankTransferPayment implements PaymentMethod {public String name(){return "Bank
Transfer";}public boolean processPayment(double a){return true;}}
class Order {enum Status{Pending,Paid}final String id;final Customer customer;private final
List<OrderItem> items=new ArrayList<>();private Status status=Status.Pending;
Order(String i,Customer c){id=i;customer=c;}void add(Product p,int
q){if(status==Status.Paid)throw new IllegalStateException("Order already paid");items.add(new
OrderItem(p,q));}
double total(){return items.stream().mapToDouble(OrderItem::total).sum();}Status
status(){return status;}
void pay(PaymentMethod m){if(items.isEmpty())throw new IllegalStateException("Cannot process
payment for an empty order.");System.out.println("Payment initiated via "+m.name()+" for Order
"+id+".");boolean
ok=m.processPayment(total());if(ok)status=Status.Paid;System.out.println("Payment for Order
"+id+(ok?" successful.":" failed."));System.out.println("Order status: "+status+".");}}
public class Main {public static void main(String[] args){Order x=new Order("X",new
Customer("Customer X"));System.out.println("Order created for Customer X.");x.add(new
Product("Product A",20),2);x.add(new Product("Product B",15),1);x.pay(new
CreditCardPayment());
Order y=new Order("Y",new Customer("Customer Y"));try{y.pay(new
CreditCardPayment());}catch(IllegalStateException e){System.out.println(e.getMessage());}
Order z=new Order("Z",new Customer("Customer Z"));System.out.println("Order created for
Customer Z.");z.add(new Product("Product C",12),1);z.pay(new PayPalPayment());}}
```

Expected: Credit Card succeeds and X becomes Paid; empty order is rejected; PayPal failure leaves Z Pending.
Payment outcomes are simulated because no real payment gateway is defined in the source.